

Лабораторная работа №5.

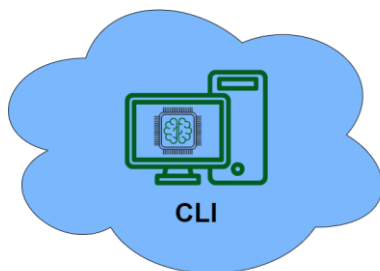
Обработка текста

Цель работы:

- Изучить основы получения документации в UNIX-подобных операционных системах

Оборудование, ПО:

- ALT1
- Справочная литература или доступ в сеть интернет.



Машина	IP	Маска	Шлюз	DNS
CLI (Альт Рабочая станция 10.4)	- 192.168.1.1	- 24	- -	- 77.88.8.8

Контрольные задания:

1. Найдите строку, содержащую слово user в файле /etc/group.
2. Отобразите все строки, содержащие слово root из файлов /etc/passwd и /etc/group.
3. Подсчитайте количество строк, со словом student в файле /etc/group.
4. Подсчитайте количество строк, в которых НЕТ слова student в файле /etc/group.
5. Используя команду grep, отобразите все строки, содержащие слово List или слово System в файле /etc/passwd.
6. Найдите в директории /usr все файлы и директории, имя которых заканчивается на li.
7. Найдите в домашней директории пользователя student только файлы.
8. Найдите в домашней директории пользователя student только директории
9. Найдите все файлы, принадлежащие пользователю student.
10. Найдите все файлы, принадлежащие группе ftp.
11. Найдите все файлы в системе размером более 200М.
12. Найдите в директории /tmp файлы нулевого размера.
13. Пройдите 1-4 главу в vimtut

Используемые источники:

- Официальная документация операционной системы Альт Рабочая Станция 10.4 <https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html/alt-workstation/index.html>

Cat

Название команды – сокращение от слова catenate (объединить). Команда для просмотра содержимого файла. Это одна из наиболее часто используемых команд в Linux.

Сначала посмотрим справку по данной команде.

```
$ man cat
```

Результат работы команды:

Тут мы можем увидеть, что данная команда предназначена для объединения файлов и печати на стандартный вывод информации. Под стандартным выводом подразумевается вывод в терминал. Так же можно увидеть, что у данной команды есть ключи.

Синтаксис команды cat:

```
cat [ОПЦИИ] ... [ФАЙЛ] ...
```

Самое простое и очевидное действие, где используется команда cat – это просмотр содержимого файла.

Выведем содержимое файла /etc/passwd:

```
$ cat /etc/passwd
```

Результат работы команды:

Можно передать утилите несколько файлов и тогда их содержимое будет выведено поочередно, без разделителей. Опции позволяют очень сильно видоизменить вывод и сделать именно то, что вам нужно. Рассмотрим основные опции:

- b нумеровать только непустые строки;
- E показывать символ \$ в конце каждой строки;
- n нумеровать все строки;
- s удалять пустые повторяющиеся строки;
- Т отображать табуляции в виде ^I;
- h отобразить справку;
- v версия утилиты.

Результат работы команды с опциями:

Если файл не помещается на один экран и прокручивается очень быстро, можно использовать параметры `more` и `less` с помощью команды `cat`.

`$ cat /etc/passwd | more`

Результат работы команды:

`$ cat /etc/passwd | less`

Результат работы команды:

Создадим файл `роем.txt` со следующим содержанием:

Ночь

Улица

Фонарь

Аптека

Аптека

Улица

Фонарь

С помощью опции `-n` пронумеруем строки файла `роем.txt`:

`$ cat -n роem.txt`

Результат работы команды:

Если необходимо пронумеровать только непустые строки, то воспользуемся опцией `-b`:

`$ cat -b роem.txt`

Результат работы команды:

Если не передать никакого файла в параметрах утилите, то она будет пытаться читать данные из стандартного ввода:

```
$ cat
```

Для завершения записи нажмите Ctrl+D.

Результат работы команды:

Head

Команда `head` в Linux выводит начальные строки (по умолчанию — 10) из одного или нескольких документов. Очень удобная команда для просмотра того, что было в начале файла.

Также она может показывать данные, которые передаёт на вывод другая утилита.

Выведем первые 10 строк файла `/etc/passwd`:

```
$ head /etc/passwd
```

Результат работы команды:

Для изменения количества выводимых строк необходимо использовать ключ `n` и за ним указать необходимое количество строк.

Выведем первые 3 строки файла `/etc/passwd`:

```
$ head -n 3 /etc/passwd
```

Результат работы команды:

или

```
$ head 3 /etc/passwd
```

Результат работы команды:

Можно передать утилите несколько файлов и тогда их содержимое будет выведено поочередно. Чтобы не возникало путаницы, их содержимое автоматически разделяется пустой строкой, а перед текстом выводится название документа.

Скопируем файл `роем2.txt` и применим команду `head` для 2-х этих файлов:

```
$ cp роем.txt роем2.txt
```

```
$ head роем.txt роем2.txt
```

Результат работы команды:

Чтобы название файла выводилось даже в том случае, когда команде задан только один файл, следует воспользоваться опцией `-v`:

```
$ head -v роем.txt
```

Результат работы команды:

Также часто применяются следующие ключи:

`-c (--bytes)` — позволяет задавать количество текста не в строках, а в байтах. При записи в виде `--bytes=[-]NUM` выводит на экран все содержимое файла, кроме `NUM` байт, расположенных в конце документа.

`-q (--quiet, --silent)` — выводит только текст, не добавляя к нему название файла.

`-z (--zero-terminated)` — символы перехода на новую строку заменяет символами завершения строк.

Tail

Команда `head` в Linux выводит последние строки (по умолчанию — 10) из одного или нескольких документов. Очень удобная команда для просмотра того, что было в конце файла.

Выведем последние 10 строк файла `/etc/passwd`:

```
$ tail /etc/passwd
```

Результат работы команды:

Для изменения количества выводимых строк необходимо использовать ключ `n` и за ним указать необходимое количество строк.

Выведем последние 3 строки файла `/etc/passwd`:

```
$ tail -n 3 /etc/passwd
```

Результат работы команды:

Для отслеживания появления новых строк в файле необходимо использовать опцию `-f`:

```
$ ping ya.ru > ping &
```

```
$ tail -f ping
```

Результат работы команды:

С помощью опции `-s` можно задать частоту обновления файл (по умолчанию данные обновляются раз в секунду).

Результат работы команды:

Можно передать утилите несколько файлов и тогда их содержимое будет выведено поочередно. Чтобы не возникало путаницы, их содержимое

автоматически разделяется пустой строкой, а перед текстом выводится название документа.

```
$ tail poem.txt poem2.txt
```

Результат работы команды:

Также часто применяются следующие ключи:

- с позволяет вывести указанное количество байт с конца файла;
- pid используется с опцией -f, позволяет завершить работу утилиты, когда завершится указанный процесс;
- q не выводит имена файлов;
- retry повторяет попытки открыть файл, если он недоступен.

Grep

Grep (General Regular Expression Print) является одной из самых востребованных команд в терминале Linux, которая входит в состав проекта GNU. До того как появился проект GNU, существовала утилита предшественник грег, с тем же названием, которая была разработана в 1973 году Кеном Томпсоном для поиска файлов по содержимому в Unix. А потом уже была разработана свободная утилита с той же функциональностью в рамках GNU.

Команда грег используется для поиска по паттернам (т.е. шаблонам) в тексте. Грег дает очень много возможностей для фильтрации текста. Вы можете выбирать нужные строки из текстовых файлов, отфильтровать вывод команд, и даже искать файлы в файловой системе, которые содержат определённые строки. Утилита очень популярна, потому что она уже предустановлена почти во всех дистрибутивах.

Примеры (в терминале)

Рассмотрим на практике основные опции утилиты grep:

1. Поиск текста в файле

В первом примере мы будем искать информацию о пользователе root в файле со списком пользователей системы /etc/passwd. Для этого выполните следующую команду:

```
$ grep root /etc/passwd
```

Результат работы команды:

С помощью опции `-i` можно указать, что регистр символов учитывать не нужно. Например, давайте найдём все строки содержащие вхождение слова `time` в том же файле:

```
$ grep -i "roOt" /etc/passwd
```

Результат работы команды:

При указании нескольких условий для поиска, используем опцию `-e`:

```
$ grep -e "root" -e "daemon" /etc/passwd
```

Результат работы команды:

С помощью опции `-n` можно выводить номер строки, в которой найдено вхождение:

```
$ grep -n "root" /etc/passwd
```

Результат работы команды:

Это всё хорошо работает пока ваш поисковый запрос не содержит специальных символов. Например, если вы попытаетесь найти все строки, которые содержат символ `"["` в файле `/etc/grub.cfg`, то получите ошибку, что это регулярное выражение не верно. Для того чтобы этого избежать, нужно явно указать, что вы хотите искать строку с помощью опции `-F`:

```
$ grep -F "[" /etc/grub.cfg
```

Результат работы команды:

2. Фильтрация вывода команды

Для того чтобы отфильтровать вывод другой команды с помощью `grep` достаточно перенаправить его используя оператор `|`. А файл для самого `grep` указывать не надо. Например, для того чтобы найти все процессы `gnome` можно использовать такую команду:

```
$ ps aux | grep "gnome"
```

Результат работы команды:

3. Базовые регулярные выражения

Утилита `grep` поддерживает несколько видов регулярных выражений. Это базовые регулярные выражения, которые используются по умолчанию и расширенные. Базовые регулярные выражение поддерживает набор символов, позволяющих описать каждый определённый символ в строке. Это: `.`, `*`, `[]`, `^`, `^` и `$`. Например, вы можете найти строки, которые начинаются на букву `г`:

```
$ grep "^г" /etc/passwd
```

Результат работы команды:

Или же строки, которые содержат большие буквы:

```
$ grep "[A-Z]" /etc/passwd
```

Результат работы команды:

А так можно найти все строки, которые заканчиваются на `bash` в файле `/etc/passwd`:

```
$ grep "bash$" /etc/passwd
```

Результат работы команды:

Однако используя базовый синтаксис, мы не можем указать точное количество этих символов.

4. Расширенные регулярные выражения

В дополнение ко всем символам из базового синтаксиса, в расширенном синтаксисе поддерживаются также такие символы:

- + одно или больше повторений предыдущего символа;
- ? ноль или одно повторение предыдущего символа;
- {n,m} повторение предыдущего символа от n до m раз;
- | позволяет объединять несколько паттернов (шаблонов).

Для активации расширенного синтаксиса нужно использовать опцию -E. Например, вместо использования опции -e, можно объединить несколько слов для поиска вот так:

```
$ grep -E "root|user" /etc/passwd
```

Результат работы команды:

5. Рекурсивный поиск в grep

До этого мы рассматривали поиск в определённом файле или выводе команд. Но grep также может выполнить поиск текста в нескольких файлах, размещённых в одном каталоге или подкаталогах. Для этого нужно использовать опцию -r. Например, давайте найдём все файлы, которые содержат строку Kernel в папке /var/log:

```
$ grep -r "Kernel" /var/log
```

Результат работы команды:

Некоторые файлы доступны только суперпользователю и для того чтобы выполнять по ним поиск вам нужно запускать grep с помощью sudo. Или же можно просто скрыть сообщения об ошибках чтения и пропускать такие файлы с помощью опции -s:

```
$ grep -rs "Kernel" /var/log
```

Результат работы команды:

--

7. Выбор файлов для поиска

С помощью опций `--include` и `--exclude` можуж фильтровать файлы, которые будут принимать участие в поиске. Например, для того чтобы выполнить поиск только по файлам с расширением `.log` в папке `/var/log` используйте такую команду:

```
$ grep -r --include="*.log" "Kernel" /var/log
```

Результат работы команды:

8. Поиск слов в grep

Когда мы ищем строку `abc`, `grep` будет выводить также `kbabc`, `abc123`, `aafrabc32` и тому подобные комбинации. Мы можем заставить утилиту искать по содержимому файлов в Linux строки, которые включают только искомые слова полностью с помощью опции `-w`:

```
$ grep -w "Ночь" poem.txt
```

(предварительно поменять немного текст в файле)

Результат работы команды:

9. Количество строк

Утилита `grep` может сообщить, сколько строк с определенным текстом было найдено файле. Для этого используется опция `-c` (счетчик).

```
$ grep -c "Аптека" poem.txt
```

Результат работы команды:

10. Инвертированный поиск

Команда `grep` Linux может быть использована для поиска строк, которые не содержат указанное слово. Например, так можно вывести только те строки, которые не содержат слово `nologin`:

```
$ grep -v nologin /etc/passwd
```

Результат работы команды:

11. Цветной вывод

По умолчанию `grep` не будет подсвечивать совпадения цветом. Но в большинстве дистрибутивов прописан алиас для `grep`, который это включает. Однако, когда вы используете команду с `sudo` это работать не будет. Для включения подсветки вручную используйте опцию `--color` со значением `always`:

```
$ grep --color=always root /etc/passwd
```

Результат работы команды:

Cut

Команда `cut` в Linux позволяет вам выбирать часть содержимого файла по байтам, по символу, по полю или по столбцу. Также команда `cut` используется, если нужно вырезать часть текста — при этом он может находиться в файле либо быть напечатанным через стандартный ввод.

Синтаксис команды `cut` выглядит следующим образом:

```
cut [ОПЦИИ] ... [ФАЙЛ...]
```

Использовать файл не обязательно. Если на месте его названия поставить прочерк - либо не указать ничего, команда возьмет текст из стандартного ввода. При необходимости можно указывать больше файлов, чем один.

Основные параметры команды `cut`:

`-b (--bytes=LIST)` — номер байта, набор или диапазон байтов, подлежащих вырезанию.

`-c (--characters=LIST)` — символ, который следует вырезать. Также можно указывать набор либо диапазон символов.

-d (--delimiter=DELIM) — с помощью этой опции пользователь устанавливает свой разделитель вместо стандартного TAB.

-f (--fields=LIST) — перечень полей для вырезания.

-s (--only-delimited) — если была применена эта опция, cut не выводит строки, где нет разделителя.

--complement — задает байты, символы или поля, которые останутся в файле или тексте из стандартного ввода. Все остальное будет вырезано.

--output-delimiter=STRING — по умолчанию выходной разделитель соответствует входному. Эта опция позволяет задать другой выходной разделитель.

-z, --zero-terminated — вместо символа новой строки разделителем будет NULL.

```
$ cat /etc/passwd | cut -d: -f1,5-7
```

Команда сначала выводит содержимое файла /etc/passwd, а затем использует cut, чтобы разделить каждую строку по символу “:”, который обычно используется для разделения полей в файле /etc/passwd. Затем выбирается первое поле (имя пользователя) и пятое, шестое и седьмое поля (информация о пользователе, такая как домашний каталог, идентификатор пользователя и идентификатор группы).

Результат работы команды:

```
$ cd /etc
```

```
$ ls -l | cut -c1-11,50-
```

Результат работы команды:

Команда сначала выводит информацию о файлах в текущем каталоге с детальным форматом (результат команды ls -l). Затем используется команда cut, чтобы выбрать символы с первого по одиннадцатый и с пятидесятого до конца каждой строки вывода.

```
$ echo "operational system" | cut -b 1
```

Результат работы команды:

Команда вырезает из текста, напечатанного через стандартный ввод, первый символ.

```
$ echo "operational system" | cut -b 5-10
```

Результат работы команды:

Команда вырезает из текста, напечатанного через стандартный ввод, диапазон символов от 5-го до 10-го символа.

Find

Команда `find` — одна из самых мощных и широко применимых команд. Утилита используется для поиска файлов и каталогов на основе различных критериев.

Общий синтаксис команды `find` выглядит так:

```
find [ПАПКА] [ОПЦИИ] [КРИТЕРИЙ] ШАБЛОН [ДЕЙСТВИЕ]
```

где ПАПКА указывает на расположение, с которого необходимо начать поиск. Поиск по умолчанию рекурсивные и начинается с текущего расположения;

КРИТЕРИЙ содержит указание типа поиска: по имени, типу файла, размеру, времени изменения и т.д.;

ШАБЛОН содержит поисковый запрос.

Примеры (в терминале)

Найдем все файлы и директории, имя которых точно совпадает с "user" в текущем каталоге (.) и его подкаталогах.

```
$ find . -name user
```

Результат работы команды:

--

Поскольку тип объекта не указан, команда выполняет поиск и файлов, и папок.

Если нужно искать только файлы, то добавляем опцию `-type f`:

```
$ find . -type f -name sshd
```

Результат работы команды:

Тип и имя можно указывать в любом порядке.

Команда `find` по умолчанию учитывает регистр. Чтобы выполнить поиск по имени файла без учёта регистра, надо ввести опцию `-iname` вместо `-name`.

Одно из самых популярных применений команды `find` — поиск файлов по заданному расширению.

Найдем все файлы в текущих папках с `.txt` расширением:

```
$ find . -type f -name "*.txt"
```

Результат работы команды:

Опция `-empty` позволяет использовать команду `find` для поиска пустых файлов и папок.

Найти таковые в текущей папке можно следующим образом:

```
$ find . -empty
```

Результат работы команды:

Команда `find` позволяет выполнить поиск по размеру. Но это работает только для файлов, не для папок! Опция `-size` с аргументом `+N` для файлов размером более `N` и `-N` для файлов размером менее `N`.

Найдем все файлы с размером больше 1 Мб в текущем каталоге и его подкаталогах

```
$ find . -size +1M
```

Результат работы команды:

Wc

Используется для подсчета количества слов, строк, символов и размера файла.

Для запуска утилиты откройте терминал и введите:

```
wc
```

Терминал будет ожидать ввода данных. После нажатия комбинации клавиш `Ctrl + D` командный интерпретатор завершит работу программы и выведет три числа, обозначающих количество строк, слов и байт введенной информации.

Утилита может обрабатывать файлы. Стандартная инструкция выглядит так:

```
wc file
```

`wc` — имя утилиты;

`file` — название обрабатываемого файла.

Программа также может принимать параметры для анализа отдельных значений.

`-l` опция для подсчета строк.

`-w` опция для подсчета слов.

`-c` опция для подсчета байт.

`-m` опция для подсчета символов.

Выведем только количество символов и строк двух файлов.

Обратите внимание: порядок указания параметров не влияет на итоговый

вид информации. Программа всегда выводит данные в виде СТРОК — СЛОВ — БАЙТ (СИМВОЛОВ) [— ФАЙЛ]. Если какой-то параметр будет отсутствовать, его столбец просто проигнорируется, не задевая остальные. Количество символов будет стоять первым, если в команде содержался и вывод байт.

```
$ wc /etc/passwd
```

Результат работы команды:

```
$ ls /etc | wc -l 142
```

Результат работы команды:

Редакторы текста

Программы редактирования текста являются необходимыми инструментами для работы с текстовыми файлами в Linux. Они позволяют создавать, редактировать и сохранять текстовые файлы для широкого круга целей, включая программирование, написание документов и редактирование конфигурационных файлов.

В Linux доступно несколько программ редактирования текста, каждая из которых имеет свои сильные и слабые стороны. Некоторые из наиболее популярных текстовых редакторов в Linux включают:

Vi/vim: Vi/vim широко используемый текстовый редактор в Linux. Это мощный и хорошо настраиваемый редактор, который предоставляет широкий спектр возможностей, включая подсветку синтаксиса, поиск и замену, запись макросов и поддержку различных языков программирования. Однако Vi/vim имеет крутую кривую обучения и может быть сложным в использовании для новичков.

nano: nano - это простой и удобный текстовый редактор, который проще в использовании, чем Vi/vim. Он предоставляет основные возможности, такие как подсветка синтаксиса, поиск и замена, а также нумерация строк. Nano - хороший выбор для начинающих или для пользователей, которым нужно быстро и легко редактировать текстовые файлы.

emacs: emacs - это мощный и расширяемый текстовый редактор, который предоставляет широкий спектр возможностей для опытных пользователей. Он включает такие функции, как подсветка синтаксиса, поиск и замена, запись макросов и поддержка различных языков программирования. Однако, как и Vi/vim, он имеет крутую кривую обучения и может быть сложным для новичков.

gedit: gedit - это графический текстовый редактор, предоставляющий удобный интерфейс для редактирования текстовых файлов. Он включает такие функции, как подсветка синтаксиса, поиск и замена, а также поддержку различных языков программирования. Gedit - хороший выбор для пользователей, предпочитающих графический интерфейс для редактирования текста.

Помимо этих текстовых редакторов, в Linux доступно еще несколько программ для редактирования текста, включая Kate, Geany и Sublime Text. Каждый из этих редакторов предоставляет уникальный набор функций и возможностей, поэтому важно выбрать тот, который лучше всего отвечает вашим потребностям.

- vi/vim
- emacs
- nano
- mcedit
- pluma

Nano

Nano – простой и удобный текстовый редактор, который широко используется в системах Linux и Unix. Он предоставляет такие базовые возможности, как подсветка синтаксиса, поиск и замена, а также нумерация строк. В этом разделе мы опишем возможности использования nano, а также приведем несколько примеров его использования.

Возможность использования nano:

Nano – текстовый редактор командной строки, который используется в системах Linux и Unix. Он прост в использовании и предоставляет простой интерфейс для редактирования текстовых файлов. Чтобы использовать nano, нужно просто открыть окно терминала и ввести "nano", а затем имя файла, который вы хотите отредактировать. Это приведет к открытию текстового редактора и отображению содержимого файла.

Nano имеет строку меню в нижней части экрана, которая отображает различные опции для редактирования и манипулирования файлом. Пользователи могут перемещаться по файлу с помощью клавиш со стрелками и переключаться между командным режимом и режимом вставки, нажимая клавишу "Ctrl".

Примеры использования:

1. Создание нового файла: Чтобы создать новый файл с помощью nano, просто введите "nano filename" в терминале. Это откроет новый файл с указанным именем в режиме вставки. Вы можете сразу же начать вводить текст.
2. Редактирование существующего файла: Чтобы отредактировать существующий файл с помощью nano, просто введите в терминале "nano filename", где "filename" - имя файла, который вы хотите отредактировать. Это откроет файл в режиме вставки. Вы можете сразу же приступить к редактированию файла.
3. Сохранение изменений: Чтобы сохранить изменения, внесенные в файл с помощью nano, переключитесь в командный режим, нажав клавишу "Ctrl", а затем введите "Ctrl+O" и затем "Enter". Это сохранит изменения, внесенные в файл.

4. Выход из nano: Чтобы выйти из nano, перейдите в командный режим, нажав клавишу "Ctrl", а затем введите "Ctrl+X". Это приведет к выходу из текстового редактора. Если вы внесли изменения в файл и хотите сохранить их перед выходом, nano предложит вам сохранить изменения.

5. Поиск текста: Чтобы найти текст в файле с помощью nano, переключитесь в командный режим, нажав клавишу "Ctrl", а затем введите "Ctrl+W". Появится запрос на ввод поискового запроса. Введите поисковый запрос и нажмите "Enter", чтобы найти его в файле.

В целом, nano - это простой и удобный текстовый редактор, который легко использовать для основных задач редактирования текста. Он идеально подходит для новичков или для пользователей, которым необходимо быстро и легко редактировать текстовые файлы. Немного попрактиковавшись, пользователи смогут освоить использование nano для решения различных задач, от создания новых файлов до поиска текста в существующих файлах.

\$ nano file.txt – команда для открытия файла file.txt в текстовом редакторе nano

Команды	Описание
---------	----------

^O	Сохранить, запросит подтверждение файла
----	---

^X	Выйти
----	-------

^W	Поиск
----	-------

^W, ^R	Поиск и замена
--------	----------------

^Y	страница вверх
----	----------------

^V	страница вниз
----	---------------

^G	справка по командам
----	---------------------

Vi/vim

Vim и Vi – текстовые редакторы командной строки, которые используются в системах Linux и Unix. Они легкие и быстрые, что делает их идеальными для редактирования больших текстовых файлов. Оба редактора используют модальный интерфейс, который позволяет пользователям переключаться между различными режимами, такими как командный режим и режим вставки, для выполнения различных задач.

Чтобы использовать Vim или Vi, нужно просто открыть окно терминала и набрать "vi" или "vim", а затем имя файла, который вы хотите отредактировать. Это откроет текстовый редактор в командном режиме, который позволяет перемещаться по файлу и выполнять различные команды.

Примеры использования:

1. Создание нового файла: Чтобы создать новый файл с помощью Vim или Vi, просто введите "vi filename" в терминале. Это откроет новый файл с указанным именем в командном режиме. Чтобы переключиться в режим вставки и начать вводить текст, нажмите клавишу "i".

2. Редактирование существующего файла: Чтобы отредактировать существующий файл с помощью Vim или Vi, просто наберите в терминале "vi filename", где "filename" - имя файла, который вы хотите отредактировать. Это откроет файл в командном режиме. Чтобы переключиться в режим вставки и начать редактирование файла, нажмите клавишу "i".

3. Сохранение изменений: Чтобы сохранить изменения, внесенные в файл с помощью Vim или Vi, переключитесь в командный режим, нажав клавишу "Esc", а затем введите ":w" и затем "Enter". Это сохранит изменения, внесенные в файл.

4. Выход из Vim или Vi: Чтобы выйти из Vim или Vi, перейдите в командный режим, нажав клавишу "Esc", а затем введите ":q", после чего нажмите "Enter". Это приведет к выходу из текстового редактора. Если вы внесли изменения в файл и хотите сохранить их перед выходом, введите ":wq", а затем "Enter".

5. Поиск текста: Чтобы найти текст в файле с помощью Vim или Vi, переключитесь в командный режим, нажав клавишу "Esc", а затем введите "/searchterm" с последующим "Enter". Это приведет к поиску указанного поискового термина в файле.

6. В целом, Vim и Vi - это мощные текстовые редакторы, предоставляющие широкий спектр функций и возможностей для редактирования и работы с текстовыми файлами в системах Linux и Unix. С некоторой практикой пользователи могут стать опытными в использовании этих редакторов для различных задач, от создания новых файлов до поиска текста в существующих файлах.

<code>\$ vim file.txt</code> – команда для открытия файла <code>file.txt</code> в текстовом редакторе <code>vim</code>
--

Командный режим – режим, в котором по умолчанию запускается и работает `vim`.

Режим редактирования

В `Vim`, после запуска в командном режиме, вы можете переключиться в режим редактирования для изменения текста. Для перехода клавиша `i`.

Комбинации клавиш командного режима

`:w`

Сохранить изменения в файле (`write`).

`:q`

Выйти из `Vim` (`quit`).

`:wq`

Сохранить изменения и выйти.

`:q!`

Выйти без сохранения

`:help <topic>`

Получить помощь по определенной теме или команде в `Vim`.

`vimtutor`

Запустить учебник по `Vim` (сначала его необходимо скачать командой:

`apt-get install vimtutor`)